

1. Project Overview

This project implements a Workout Tracker application using an n-tier architecture. The system was developed across multiple project stages using generative AI assistance to help generate and refine the code. The application allows users to manage workout data including users, exercises, and workouts.

The purpose of Project 4 is to perform a full system test of the entire application and verify that all layers of the system work correctly together. The application consists of a PostgreSQL database (data layer), a business logic layer implemented through service classes, a FastAPI service layer that exposes REST API endpoints, and a web client that allows users to interact with the system.

The final system supports retrieving records, inserting new records, and updating existing records through the client interface. All functionality is verified through testing and database validation.

2. System Architecture

This application follows a layered n-tier architecture consisting of four primary layers: the data layer, business layer, service layer, and client layer.

Data Layer

The data layer is implemented using PostgreSQL. It stores all application data related to users, exercises, workouts, workout exercises, and sets. Repository classes located in the `repositories` folder execute SQL queries to retrieve, insert, update, and delete data from the database. These repositories provide a clean interface for interacting with the database while keeping SQL logic separated from other parts of the application.

Business Layer

The business layer is implemented using service classes located in the `services` folder. These classes contain the business logic of the application. The services validate input data and call the repository methods in the data layer. This layer ensures that only valid

data is passed to the database and helps maintain separation of responsibilities within the system.

Service Layer

The service layer is implemented using FastAPI. The FastAPI server exposes REST API endpoints that allow the client layer to interact with the system. These endpoints allow operations such as retrieving users, creating exercises, updating workouts, and viewing workout data. The service layer acts as the interface between the client application and the business logic.

Client Layer

The client layer is a web-based front-end application that allows users to interact with the system. The client is hosted by the FastAPI application and can be accessed through a web browser. The client interface allows users to retrieve records, create new records, and update existing records for users, exercises, and workouts. The client communicates with the service layer by making HTTP requests to the API endpoints.

3. Deployment Instructions

This section explains how to download, configure, build, and run the application from the GitHub repository.

Prerequisites

Before running the project, the following software must be installed:

- Python 3.11 or later
- PostgreSQL
- pip (Python package manager)
- A code editor such as Visual Studio Code
- Git (optional, for cloning the repository)

Step 1: Download the Project

The project can be downloaded from the GitHub repository.

Option 1: Clone the repository using Git.

```
git clone <repository_url>
```

Option 2: Download the ZIP file from GitHub and extract it.

After downloading, open the project folder in your code editor.

Step 2: Install Project Dependencies

Navigate to the `project-1` folder and install the required Python packages.

```
pip install -r requirements.txt
```

This installs all required dependencies such as FastAPI, Uvicorn, Requests, and PostgreSQL database drivers.

Step 3: Configure the Database

The application uses PostgreSQL for data storage. The database connection settings are configured in the file:

```
project-1/app/db.py
```

The application can use environment variables for database configuration, including:

- PGHOST
- PGPORT
- PGDATABASE
- PGUSER
- PGPASSWORD

If these variables are not set, the default values defined in the `db.py` file will be used.

Step 4: Create and Seed the Database

The database schema and initial data are provided in the SQL scripts located in the `project-1/sql` folder.

Run the following SQL files in PostgreSQL:

`01_create_tables.sql`

`02_seed_data.sql`

These scripts create the necessary tables and insert initial sample data used by the application.

Step 5: Start the Backend Service

To start the FastAPI service layer, navigate to the `project-1` directory and run the following command:

```
python -m uvicorn app.api.server:app --reload
```

The server will start and host the API service locally.

The API service will be available at:

<http://127.0.0.1:8000>

API documentation is available through the Swagger interface at:

<http://127.0.0.1:8000/docs>

Step 6: Launch the Client Application

The web client is hosted by the FastAPI application. Once the server is running, open a web browser and navigate to:

<http://127.0.0.1:8000>

The Workout Tracker Web Client interface will load, allowing users to interact with the application.

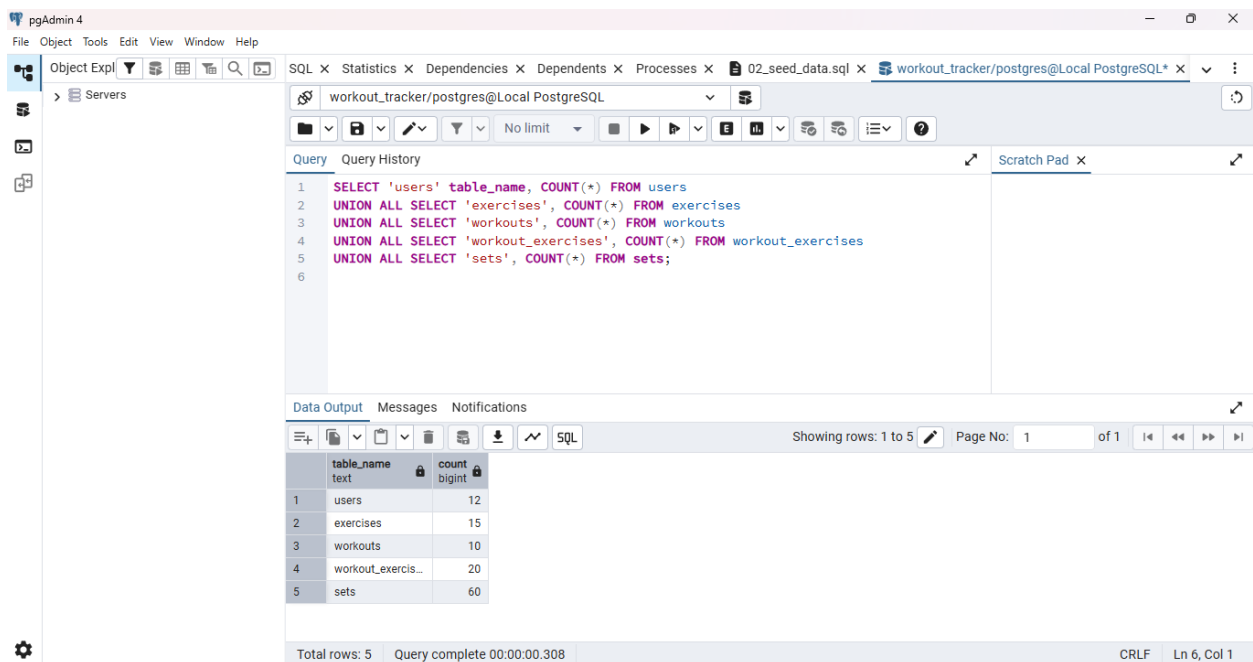
Step 7: Verify Successful Setup

The system setup is successful if the following conditions are met:

- The FastAPI server starts without errors
- The web client loads successfully in the browser
- The health check returns a successful status
- Users, exercises, and workouts can be retrieved through the interface
- New records can be created and existing records can be updated
- Database queries confirm that inserted and updated records are stored in PostgreSQL

Full System Test

1. Database Tables



The screenshot shows the pgAdmin 4 interface. The SQL editor contains a query that counts the number of records in five tables: users, exercises, workouts, workout_exercises, and sets. The query is as follows:

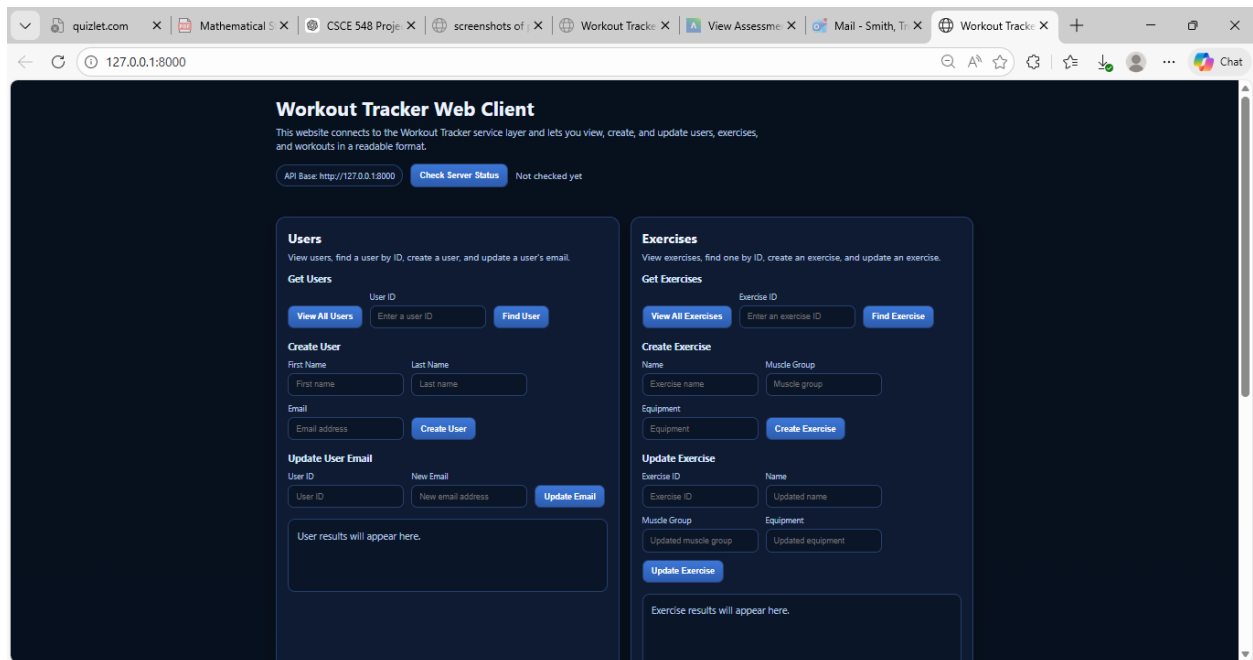
```
1 SELECT 'users' table_name, COUNT(*) FROM users
2 UNION ALL SELECT 'exercises', COUNT(*) FROM exercises
3 UNION ALL SELECT 'workouts', COUNT(*) FROM workouts
4 UNION ALL SELECT 'workout_exercises', COUNT(*) FROM workout_exercises
5 UNION ALL SELECT 'sets', COUNT(*) FROM sets;
6
```

The Data Output pane shows the results of the query, which are displayed in a table with two columns: table_name and count. The results are as follows:

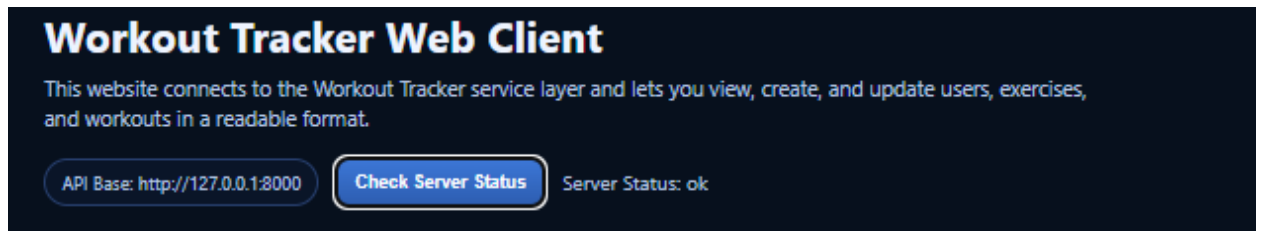
table_name	count
users	12
exercises	15
workouts	10
workout_exercises	20
sets	60

The status bar at the bottom indicates that the query is complete and that there are 5 rows in total.

2. Web Client Running



3. Health Status



User Deployment Creation/Update Test

Users

View users, find a user by ID, create a user, and update a user's email.

Get Users

User ID

View All Users

Enter a user ID

Find User

Create User

First Name

Last Name

Test

user

Email

testuser458@gmail.com

Create User

Update User Email

User ID

New Email

User ID

New email address

Update Email

User Id	First Name	Last Name	Email	Created At
23	Test	user	testuser458@gmail.com	2026-03-16T18:17:42.319389

4. Created User showing in pgAdmin

Query

Query History

1

SELECT * FROM users ORDER BY user_id DESC;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 13

Page No

	user_id [PK] integer	first_name character varying (50)	last_name character varying (50)	email character varying (255)	created_at timestamp without time zone
1	23	Test	user	testuser458@gmail.com	2026-03-16 18:17:42.319389
2	13	Project2	Test	p2test@example.com	2026-02-23 15:04:05.528938
3	11	Tre	Smith	tres@email.edu	2026-02-03 23:13:54.044047
4	10	User10	Test10	user10@example.com	2026-02-03 22:03:09.321775
5	9	User9	Test9	user9@example.com	2026-02-03 22:03:09.321775
6	8	User8	Test8	user8@example.com	2026-02-03 22:03:09.321775
7	7	User7	Test7	user7@example.com	2026-02-03 22:03:09.321775
Total rows: 13		Query complete 00:00:01.029			

5. Update User

Users

View users, find a user by ID, create a user, and update a user's email.

Get Users

User ID

View All Users

Enter a user ID

Find User

Create User

First Name

Last Name

Email

Test

user

testuser458@gmail.com

Create User

Update User Email

User ID

New Email

23

updated@email.com

Update Email

User Id	First Name	Last Name	Email	Created At
23	Test	user	updated@email.com	2026-03-16T18:17:42.319389

6. Updated Email in pgAdmin

Query Query History

```
1 SELECT * FROM users WHERE email = 'updated@email.com';
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No

	user_id [PK] integer	first_name character varying (50)	last_name character varying (50)	email character varying (255)	created_at timestamp without time zone
1	23	Test	user	updated@email.com	2026-03-16 18:17:42.319389

Viewing All Users (GET) and Finding User

Users
View users, find a user by ID, create a user, and update a user's email.

Get Users

User ID

Create User

First Name Last Name

Email

Update User Email

User ID New Email

User Id	First Name	Last Name	Email	Created At
1	User1	Test1	user1@example.com	2026-02-03T22:03:09.321775
2	User2	Test2	user2@example.com	2026-02-03T22:03:09.321775
3	User3	Test3	user3@example.com	2026-02-03T22:03:09.321775
4	User4	Test4	user4@example.com	2026-02-03T22:03:09.321775
5	User5	Test5	user5@example.com	2026-02-03T22:03:09.321775

Users
View users, find a user by ID, create a user, and update a user's email.

Get Users

User ID

Create User

First Name Last Name

Email

Update User Email

User ID New Email

User Id	First Name	Last Name	Email	Created At
4	User4	Test4	user4@example.com	2026-02-03T22:03:09.321775

Exercise Creation/Update Deployment Test

Exercises

View exercises, find one by ID, create an exercise, and update an exercise.

Get Exercises

Exercise ID

View All Exercises

Enter an exercise ID

Find Exercise

Create Exercise

Name

Muscle Group

Test

Chest

Equipment

Barbell

Create Exercise

Update Exercise

Exercise ID

Name

Exercise ID

Updated name

Muscle Group

Equipment

Updated muscle group

Updated equipment

Update Exercise

Exercise Id	Name	Muscle Group	Equipment
19	Test	Chest	Barbell

Created Exercise in pgAdmin

Query

Query History

1

```
SELECT * FROM exercises ORDER BY exercise_id DESC;
```

Cancel query

Alt

Shift

C

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 16

Page

	exercise_id [PK] integer	name character varying (100)	muscle_group character varying (50)	equipment character varying (50)
1	19	Test	Chest	Barbell
2	15	Plank	Core	Bodyweight
3	14	Calf Raise	Legs	Machine
4	13	Deadlift	Legs	Barbell
5	12	Leg Press	Legs	Machine
6	11	Squat	Legs	Barbell
7	10	Lateral Raise	Shoulders	Dumbbell

Total rows: 16

Query complete 00:00:00.285

7. Updated Exercise in Client

Exercises

View exercises, find one by ID, create an exercise, and update an exercise.

Get Exercises

Exercise ID

View All Exercises

Enter an exercise ID

Find Exercise

Create Exercise

Name

Muscle Group

Test

Chest

Equipment

Barbell

Create Exercise

Update Exercise

Exercise ID

Name

19

updated exercise

Muscle Group

Equipment

Chest

Dumbbell

Update Exercise

Exercise Id	Name	Muscle Group	Equipment
19	updated exercise	Chest	Dumbbell

8. Updated Exercise in pgAdmin

Query Query History

```
1 SELECT * FROM exercises WHERE exercise_id = 19;
```

Data Output Messages Notifications

Showing

	exercise_id [PK] integer	name character varying (100)	muscle_group character varying (50)	equipment character varying (50)
1	19	updated exercise	Chest	Dumbbell

Viewing All Exercises and Finding Exercise (GET)

Exercises

View exercises, find one by ID, create an exercise, and update an exercise.

Get Exercises

Exercise ID

View All Exercises Find Exercise

Create Exercise

Name Muscle Group

Equipment Create Exercise

Update Exercise

Exercise ID Name

Muscle Group Equipment

Update Exercise

Exercise Id	Name	Muscle Group	Equipment
1	Bench Press	Chest	Barbell
2	Incline DB Press	Chest	Dumbbell
3	Push-Up	Chest	Bodyweight
4	Pull-Up	Back	Bodyweight
5	Lat Pulldown	Back	Machine
6	Barbell Row	Back	Barbell

Exercises

View exercises, find one by ID, create an exercise, and update an exercise.

Get Exercises

Exercise ID

View All Exercises Find Exercise

Create Exercise

Name Muscle Group

Equipment Create Exercise

Update Exercise

Exercise ID Name

Muscle Group Equipment

Update Exercise

Exercise Id	Name	Muscle Group	Equipment
7	Bicep Curl	Arms	Dumbbell

Workout Creation/Update Deployment Test

Workouts

View all workouts, find one by ID, view workouts for a user, create a workout, and update a workout.

Get Workouts

View All Workouts

Workout ID
Enter a workout ID
Find Workout

User ID (Workout Subset)
Enter a user ID
View Workouts for User

Create Workout

User ID
1

Workout Date
03/16/2026

Workout Name
demo workout

Notes
testing

Create Workout

Update Workout

Workout ID
Workout ID

Workout Date
mm/dd/yyyy

Workout Name
Updated workout name

Notes
Updated notes

Update Workout

Workout Id	User Id	Workout Date	Name	Notes
34	1	2026-03-16	demo workout	testing

Created Workout in pgAdmin

Query

Query History

1SELECT * FROM workouts WHERE workout_id = (34);

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 1

Page

	workout_id [PK] integer	user_id integer	workout_date date	name character varying (100)	notes text	created_at timestamp without time zone
1	34	1	2026-03-18	new workout	boomsti...	2026-03-16 18:29:56.761487

Viewing all Workouts and Finding Workouts (GET)

Workouts

View all workouts, find one by ID, view workouts for a user, create a workout, and update a workout.

Get Workouts

Workout ID

View All Workouts

Enter a workout ID

Find Workout

User ID (Workout Subset)

Enter a user ID

View Workouts for User

Create Workout

User ID

1

Workout Date

03/16/2026

Workout Name

demo workout

Notes

testing

Create Workout

Update Workout

Workout ID

34

Workout Date

03/18/2026

Workout Name

new workout

Notes

boomstick

Update Workout

Workout Id	User Id	Workout Date	Name	Notes
34	1	2026-03-18	new workout	boomstick
28	10	2026-02-03	Pull Day	Auto-generated workout
1	1	2026-02-02	Pull Day	Auto-generated workout
4	2	2026-02-01	Leg Day	Auto-generated workout
7	3	2026-01-31	Push Day	Auto-generated workout
10	4	2026-01-30	Pull Day	Auto-generated workout
13	5	2026-01-29	Leg Day	Auto-generated workout
16	6	2026-01-28	Push Day	Auto-generated workout
19	7	2026-01-27	Pull Day	Auto-generated workout

Workouts

View all workouts, find one by ID, view workouts for a user, create a workout, and update a workout.

Get Workouts

View All Workouts

Workout ID

Enter a workout ID

Find Workout

User ID (Workout Subset)

4

View Workouts for User

Create Workout

User ID

1

Workout Date

03/16/2026

Workout Name

demo workout

Notes

testing

Create Workout

Update Workout

Workout ID

34

Workout Date

03/18/2026

Workout Name

new workout

Notes

boomstick

Update Workout

Workout Id	User Id	Workout Date	Name	Notes
10	4	2026-01-30	Pull Day	Auto-generated workout